ANALITYKA GOSPODARCZA / DATA MINING

1. Notebook i kolejność wykonywania

Notebook składa się z komórek kodu oraz komórek tekstowych. Kod wykonuje się w bieżącej sesji Pythona.

```
liczba = 8
liczba * 3
```

- W komórce można umieścić kilka instrukcji, a ostatnie wyrażenie często jest wyświetlane jako wynik.
- Zmienne utworzone w jednej komórce mogą zostać wykorzystane w następnej.
- Wynik zależy od kolejności uruchamiania komórek, nie wyłącznie od ich położenia na ekranie.

**Kiedy używać:** Zaczynając pracę w notatniku, uruchom komórki kolejno od góry do dołu.

**Na co uważać:** Po ponownym uruchomieniu kernela zmienne znikają. Gdy widzisz NameError mimo poprawnego kodu, sprawdź, czy wykonano wcześniejszą komórkę.

## 2. Komentarze, nazwy i przypisanie

Znak # rozpoczyna komentarz. Znak = przypisuje wartość do nazwy, nie sprawdza równości.

```
liczba_transakcji = 120  # liczba w badanym tygodniu
srednia_wartosc = 56.5
print(liczba_transakcji, srednia_wartosc)
```

- Po lewej stronie = znajduje się nazwa zmiennej, po prawej wyrażenie.
- Przyjęta konwencja to małe litery i podkreślenia, np. wartosc_netto.
- Python rozróżnia wielkie i małe litery: wynik i Wynik to dwie różne nazwy.

**Kiedy używać:** Gdy chcesz przechować wynik obliczenia do dalszego wykorzystania.

**Na co uważać:** Nie zapisuj nazw z odstępami ani słów zarezerwowanych, np. for lub if. Zwykłe = nie tworzy stałej wartości: zmienną można przypisać ponownie.

## 3. Podstawowe typy wartości

Typ określa rodzaj wartości i to, jakie operacje mają sens. Funkcja type() pozwala sprawdzić typ.

```
ilosc = 18                    # int: liczba całkowita
cena = 29.90                  # float: liczba zmiennoprzecinkowa
produkt = "notes"              # str: napis
dostepny = True               # bool: prawda/fałsz
brak = None                   # brak przypisanej wartości
print(type(cena))
```

- Napisy ujmujemy w cudzysłowy pojedyncze lub podwójne.
- True i False piszemy wielką literą.
- None jest specjalną wartością Pythona i nie oznacza ani zera, ani pustego napisu.

**Kiedy używać:** Przed obliczeniami i filtrowaniem warto rozpoznać typ wartości.

**Na co uważać:** Liczba 42 oraz napis "42" nie są tym samym. Python zwykle nie dodaje ich bez jawnej konwersji.

## 4. Działania matematyczne i kolejność obliczeń

Operatory pozwalają tworzyć wyrażenia liczbowe. Nawiasy wskazują kolejność działań.

```
przychod = 5 * 49.90
marza = (przychod - 130) / przychod
reszta = 17 % 5
pelne_paczki = 17 // 5
kwadrat = 4 ** 2
```

- + dodawanie, - odejmowanie, * mnożenie, / dzielenie.
- // oznacza dzielenie całkowite, % resztę z dzielenia, a ** potęgowanie.
- Zapis x += 1 to skrót od x = x + 1.

**Kiedy używać:** Przy liczeniu wartości, udziałów, procentów, wskaźników i zmian okres do okresu.

**Na co uważać:** Dzielenie przez 0 powoduje ZeroDivisionError. Liczby float mogą mieć niedokładną reprezentację dziesiętną.

## 5. Porównania i wartości logiczne

Porównanie zwraca True albo False. Najczęstsze operatory to ==, !=, <, >, <= i >=.

```
wartosc = 220
print(wartosc >= 200)      # True
print(wartosc == 100)      # False
print(wartosc != 0)        # True
```

- == sprawdza równość, != nierówność.
- and wymaga, by oba warunki były prawdziwe; or, by co najmniej jeden był prawdziwy.
- not odwraca wartość logiczną.

**Kiedy używać:** W warunkach i w sprawdzaniu reguł jakości.

**Na co uważać:** Nie pomyl = (przypisanie) z == (porównanie). W pandas do łączenia masek używa się & i |, a nie and i or.

## 6. Napisy i formatowanie wyników

Napisy można łączyć, zmieniać i formatować. f-string wstawia wartość zmiennej do tekstu.

```
miasto = "Gdynia"
liczba = 17
print(miasto.upper())
print(miasto.lower())
print(f"Oddział: {miasto}; liczba zamówień: {liczba}")
print(f"Średnia: {18.456:.2f} zł")
```

- .upper() i .lower() zmieniają wielkość liter w napisie.
- f przed cudzysłowem pozwala umieszczać wyrażenia w nawiasach klamrowych.
- :.2f wyświetla liczbę z dwiema cyframi po przecinku.

**Kiedy używać:** Do opisowych komunikatów, drukowania wyników i etykiet wykresów.

**Na co uważać:** Formatowanie liczby do dwóch miejsc nie zmienia automatycznie wartości przechowywanej w zmiennej.

## 7. Listy: kolejność i indeksowanie

Lista przechowuje wiele wartości w ustalonej kolejności. Indeksy zaczynają się od 0.

```
kwoty = [25, 70, 15, 90]
print(kwoty[0])          # 25
print(kwoty[-1])         # 90
print(kwoty[1:3])        # [70, 15]
kwoty.append(45)
print(len(kwoty))
```

- kwoty[0] to pierwszy element; [-1] to ostatni.
- Wyrażenie [1:3] obejmuje indeks 1, lecz nie obejmuje indeksu 3.
- append dodaje element do końca, len zwraca liczbę elementów.

**Kiedy używać:** Do niewielkich zbiorów elementów, nazw kolumn i list parametrów.

**Na co uważać:** Nieistniejący indeks powoduje IndexError. Lista może zawierać mieszane typy, ale nie zawsze jest to dobry pomysł.

## 8. Słowniki, krotki i zbiory

Różne kolekcje rozwiązują różne zadania: słownik łączy klucze z wartościami, krotka jest niemodyfikowalna, zbiór przechowuje unikalne elementy.

```
oceny = {"sklep_A": 4.8, "sklep_B": 4.1}
print(oceny["sklep_A"])
zakres = (1, 5)
kanaly = {"online", "salon", "online"}
print(kanaly)             # bez powtórzeń
```

- Słownik zapisujemy jako {klucz: wartość}. Nową parę dodajemy przez przypisanie.
- Krotkę zapisujemy zwykle w nawiasach okrągłych i nie zmieniamy jej elementów.
- Zbiór jest przydatny do sprawdzania unikalnych kategorii.

**Kiedy używać:** Słowniki pomagają przechowywać mapowania, konfiguracje oraz dzienniki pomiarów.

**Na co uważać:** Dostęp do nieistniejącego klucza słownika przez nawiasy kwadratowe powoduje KeyError. Można użyć .get().

## 9. Warunek if, elif, else

Warunek kieruje wykonanie programu do odpowiedniej gałęzi. Dwukropek rozpoczyna blok, a wcięcie wskazuje, co do niego należy.

```
czas_dostawy = 4
if czas_dostawy <= 2:
    status = "szybka"
elif czas_dostawy <= 5:
    status = "standardowa"
else:
    status = "opóźniona"
print(status)
```

- Python standardowo używa czterech spacji wcięcia.
- elif oznacza dodatkowy warunek; else działa, jeśli żaden wcześniejszy warunek nie był prawdziwy.

- Warunki sprawdzane są od góry.

**Kiedy używać:** Przy klasyfikowaniu wartości i zapisie reguł biznesowych.

**Na co uważać:** Nie pomijaj dwukropka. Wszystkie instrukcje danej gałęzi muszą mieć spójne wcięcie.

## 10. Pętla for i funkcja range()

Pętla for przechodzi po elementach kolekcji lub zakresie liczb.

```
regiony = ["północ", "południe", "centrum"]
for region in regiony:
    print(region)
for i in range(3):
    print(i)                # 0, 1, 2
```

- Nazwa po for jest zmienną bieżącego elementu.
- range(3) daje liczby od 0 do 2.
- enumerate pozwala uzyskać jednocześnie numer i wartość elementu.

**Kiedy używać:** Gdy tę samą operację trzeba wykonać dla wielu kolumn lub kategorii.

**Na co uważać:** Pętla nie jest zawsze potrzebna w pandas: często szybciej i czytelniej działa jedna operacja na całej kolumnie.

## 11. Pętla while i sterowanie pętlą

while powtarza blok, dopóki warunek pozostaje prawdziwy.

```
stan = 3
while stan > 0:
    print("Pozostało:", stan)
    stan -= 1
```

- W każdej iteracji trzeba zwykle zmienić stan lub zapewnić możliwość zakończenia.
- break przerywa pętlę, continue pomija resztę bieżącej iteracji.

**Kiedy używać:** Gdy liczba powtórzeń zależy od przebiegu obliczeń, a nie od gotowej listy.

**Na co uważać:** Brak zmiany warunku może spowodować nieskończoną pętlę.

## 12. Funkcje: def, parametry i return

Funkcja nadaje nazwę fragmentowi logiki. Przyjmuje argumenty i może zwracać wynik.

```
def wartosc_po_rabacie(cena, rabat_proc=0):
    wynik = cena * (1 - rabat_proc / 100)
    return round(wynik, 2)

print(wartosc_po_rabacie(120, 15))
```

- def rozpoczyna definicję, a nazwy w nawiasach to parametry.
- Parametr rabat_proc=0 ma wartość domyślną.
- return oddaje obliczony wynik; samo print() tylko go wyświetla.

**Kiedy używać:** Gdy ta sama procedura jest potrzebna kilkakrotnie, np. do czyszczenia zbioru danych.

**Na co uważać:** Funkcja bez return zwykle zwraca None. Nazwy zdefiniowane wewnątrz funkcji są zazwyczaj lokalne.

## 13. Import bibliotek i skróty nazw

Moduły rozszerzają możliwości Pythona. Import udostępnia ich zawartość.

```
import math
print(math.sqrt(81))

import pandas as pd
from pathlib import Path
plik = Path("sprzedaz.csv")
```

- import math pozwala używać funkcji przez math.nazwa.
- as pd tworzy ustalony skrót dla pandas.
- from pathlib import Path importuje konkretną klasę bez konieczności pisania pathlib.Path.

**Kiedy używać:** Przy analizie tabel, wykresach, plikach i obliczeniach matematycznych.

**Na co uważać:** Nazwa biblioteki musi być zainstalowana w środowisku. ModuleNotFoundError oznacza zwykle brak pakietu lub niewłaściwe środowisko.

## 14. Funkcje wbudowane oraz metody

Funkcja może działać samodzielnie, a metoda jest wywoływana po kropce na konkretnym obiekcie.

```
ceny = [10, 20, 30]
print(len(ceny))
print(sum(ceny))
nazwa = "  Salon Centralny  "
print(nazwa.strip().lower())
```

- len(), sum(), print(), int(), float(), str() to przykłady funkcji wbudowanych.
- .strip() i .lower() to metody napisów.
- Kolejne metody można łączyć, gdy wynik wcześniejszej ma odpowiedni typ.

**Kiedy używać:** Podczas czytania zapisu obiekt.metoda(argument).

**Na co uważać:** Nawiasy są istotne: .sum oznacza odwołanie do metody, a .sum() jej wywołanie.

## 15. Konwersje typów i sprawdzanie danych

Konwersja pozwala zmienić reprezentację, gdy jest to uzasadnione.

```
tekst = "125"
liczba = int(tekst)
kwota = float("12.5")
wynik = str(liczba) + " szt."
print(liczba + 5, kwota, wynik)
```

- int() tworzy liczbę całkowitą, float() liczbę zmiennoprzecinkową, str() napis.
- type() pomaga sprawdzić, czy konwersja się powiodła.
- W tabelach może wystąpić wartość brakująca; nie zawsze wolno ją bezpośrednio zamienić na int.

**Kiedy używać:** Kiedy dane przychodzą z CSV i mają niepoprawny lub nieoczekiwany typ.

**Na co uważać:** int("12.5") wywoła ValueError. Najpierw rozpoznaj, jak zapisano liczby i braki.

## 16. Operatory in i not in

Operatory sprawdzają przynależność wartości do kolekcji.

```
statusy = ["wysłane", "dostarczone", "anulowane"]
print("wysłane" in statusy)
print("zwrot" not in statusy)

rekord = {"id": 7, "kwota": 90}
print("kwota" in rekord)
```

- Dla listy sprawdzają wartości, dla słownika sprawdzają klucze.
- Zapis not in jest czytelniejszy niż negowanie całego wyrażenia.

**Kiedy używać:** Do weryfikowania dopuszczalnych kategorii lub tego, czy kolumna jest obecna w zbiorze.

**Na co uważać:** Nie myl sprawdzania wartości w zwykłej liście z filtrowaniem wielu wierszy w pandas.

## 17. Wyrażenia listowe

Wyrażenie listowe tworzy nową listę na podstawie istniejącej.

```
ceny = [80, 120, 45, 200]
duze = [cena for cena in ceny if cena >= 100]
print(duze)
```

- Część przed for określa, co trafi do wyniku.
- for wskazuje źródło elementów, a końcowe if może je odfiltrować.
- W analizie danych takie konstrukcje przydają się przy wyborze nazw kolumn.

**Kiedy używać:** Gdy potrzebujesz zwięzłego stworzenia nowej listy.

**Na co uważać:** Nie twórz nadmiernie długich, zagnieżdżonych wyrażeń. Czytelność jest ważniejsza od skrótu.

## 18. Błędy i samodzielna diagnoza

Komunikat błędu zawiera typ i często numer linii. Czytaj go od dołu oraz sprawdzaj, która operacja nie zadziałała.

```
wartosci = [10, 20]
# print(wartosci[5])        # IndexError
# print(nieistniejaca)       # NameError
# print(10 / 0)              # ZeroDivisionError
```

- SyntaxError: niepoprawny zapis kodu, często brakujący dwukropek lub nawias.
- IndentationError: nieprawidłowe wcięcie.
- TypeError: operacja niezgodna z typami; ValueError: wartość nie nadaje się do użycia.
- KeyError: nieistniejący klucz; FileNotFoundError: błędna ścieżka do pliku.

**Kiedy używać:** Gdy program przestaje działać lub zwraca nieoczekiwany wynik.

**Na co uważać:** Najpierw obejrzyj typy oraz mały fragment danych. Nie zamieniaj błędu na arbitralną wartość bez zrozumienia przyczyny.

## 19. Jak czytać złożone instrukcje

Długi zapis najlepiej rozłożyć na etapy: obiekt, wybór danych, metoda, argumenty i wynik.

```
sprzedaz = [40, 80, 20, 120]
prog = 50
wybrane = [x for x in sprzedaz if x > prog]
wynik = round(sum(wybrane) / len(wybrane), 1)
print(wynik)
```

- Najpierw powstaje lista wybranych wartości.
- sum() dodaje wybrane kwoty, len() zlicza elementy.
- round(..., 1) zaokrągla wynik do jednego miejsca po przecinku.

**Kiedy używać:** Przy analizie kodu łączącego kilka operacji.

**Na co uważać:** Jeśli lista wybrane będzie pusta, dzielenie przez len(wybrane) spowoduje błąd. Sprawdź warunki brzegowe.

## 20. Szybka ściąga ze znaków

| Zapis | Znaczenie | Przykład |
|---|---|---|
| = | przypisanie | x = 12 |
| == | równość | x == 12 |
| != | nierówność | x != 0 |
| [] | lista lub wybór elementu | ceny[0] |
| {} | słownik, zbiór albo miejsce w f-stringu | {"a": 1} |
| () | wywołanie funkcji i grupowanie | print(x) |
| : | początek bloku / fragment wycinka | if x > 0: |
| . | metoda lub atrybut | tekst.strip() |
| # | komentarz | # uwaga |
| None | brak przypisanej wartości | wynik = None |
| and / or / not | operacje logiczne | a > 0 and b > 0 |

## 22. try i except: kontrolowana obsługa błędów

Nie każdy błąd oznacza konieczność przerwania całego programu. Blok try pozwala podjąć próbę wykonania operacji, a except obsłużyć określony rodzaj wyjątku. Nie jest to sposób na ukrywanie błędów: trzeba wiedzieć, jaka sytuacja jest dopuszczalna i co zrobić, gdy wystąpi.

```
tekst = "brak"
try:
    sztuki = int(tekst)
except ValueError:
    sztuki = None
print(sztuki)
```

- Instrukcje w try uruchamiają się jako pierwsze. Jeśli int() nie potrafi odczytać napisu jako liczby całkowitej, pojawia się ValueError.
- Tylko w przypadku takiego wyjątku wykonywany jest blok except ValueError.
- Zwrócona wartość None nie jest poprawną liczbą. Należy później zdecydować, czy i jak taki brak uwzględnić w analizie.
- Nie używaj bez potrzeby except: bez nazwy błędu, ponieważ może ukrywać problemy niezwiązane z zamierzoną operacją.

## 23. None, NaN i puste napisy to nie to samo

W praktycznej analizie danych braki przyjmują kilka postaci. W zwykłym Pythonie często spotkasz None, w pandas wartości NaN i NaT, a w plikach CSV czasem pusty napis. Pusty napis jest nadal wartością tekstową. Samo sprawdzenie równości z pustym napisem nie wychwyci wszystkich braków.

```
import pandas as pd
wartosci = pd.Series([12, None, "", 0], dtype="object")
print(wartosci.isna())
print(wartosci.eq(""))
```

Pierwszy wynik wskazuje faktyczne braki, a drugi puste napisy. W analizie trzeba ustalić, czy puste napisy należy traktować jako brak. Liczba 0 nie oznacza braku: może być w pełni poprawną wartością, np. liczbą reklamacji.